Skills que uso no meu Codex

Guia de seleção e instalação para Codex e Claude Code

Atualizado em 15 de outubro de 2026

Estas são as skills que estão instaladas no meu Codex para apoiar o trabalho com sites, CRM e plataforma. Eu as selecionei para melhorar design, segurança, manutenção e continuidade das tarefas. O conjunto reúne nove escolhas principais e três skills de apoio à arquitetura. A instalação permite usá-las; a eficácia de cada uma precisa ser observada nas tarefas reais.

O que você encontra aqui

Doze fichas com finalidade, link, popularidade do repositório, classificação e exemplos. O roteiro de instalação distingue a estrutura comum das configurações que mudam entre agentes.

Como interpretar a classificação

Alta Fonte institucional e escopo claro, com conteúdo inspecionado. Quando há um teste executado, a ficha informa exatamente qual foi.

Média Projeto comunitário, adaptação local ou fluxo que depende de configuração e avaliação no ambiente de destino.

Baixa Fonte pouco verificável, instruções vagas ou incompatibilidade não resolvida. Nenhuma das doze recebeu esta classificação nesta seleção.

Classificação editorial de confiança para adoção. Ela considera procedência, clareza, compatibilidade e limites verificados; não reproduz automaticamente as notas do guia original e não certifica segurança.

Como interpretar as estrelas

São números do repositório inteiro, obtidos pela API pública do GitHub em 15/10/2026, por volta de 15h34 de Brasília. A mesma contagem se repete quando várias skills pertencem ao mesmo projeto. Popularidade não comprova segurança ou eficácia.

O recorte é esta seleção instalada. Outros plugins e skills já disponíveis na máquina não entram na contagem de doze. Claude significa Claude Code; uma instalação local não sincroniza automaticamente com a nuvem.

Relação das skills instaladas

Skill	Área	Papel	Estrelas	Classe
Frontend Design	Design de interfaces	Principal	181.353	Alta
Web Design Guidelines	UX e acessibilidade	Principal	32.476	Alta
Webapp Testing	Testes funcionais	Principal	181.353	Alta
Planning With Files	Planejamento e contexto	Principal	27.466	Média
Supply Chain Risk Auditor	Segurança de dependências	Principal	7.568	Média
Improve Codebase Architecture	Arquitetura e manutenção	Principal	292.556	Média
Karpathy Guidelines	Qualidade de implementação	Principal	220.041	Média
Caveman	Concisão de respostas	Principal	112.358	Média
Git Guardrails para Codex	Proteção do trabalho no Git	Principal	Não se aplica	Média
Codebase Design	Apoio de arquitetura	Dependência	292.556	Média
Domain Modeling	Terminologia e decisões	Dependência	292.556	Média
Grilling	Revisão de decisões	Dependência	292.556	Média

Nove escolhas principais e três dependências

Codebase Design, Domain Modeling e Grilling foram instaladas porque o fluxo de Improve Codebase Architecture as utiliza. Elas não representam três novas escolhas independentes do ranking inicial.

Git Guardrails é uma adaptação local

A adaptação está incluída no kit, sem estrelas próprias e sem URL pública. O link da ficha identifica a fonte que inspirou o trabalho, e não um repositório desta versão.

Fichas das skills

01 Frontend Design

Design de interfaces | Principal | Classificação Alta | Estrelas 181.353

Nome técnico `frontend-design`

anthropics/skills

Para que serve Orienta a criação de interfaces com identidade visual própria. Trabalha direção estética, tipografia, paleta, hierarquia e composição, com atenção à legibilidade e ao uso real da tela.

Aplicação no meu fluxo Sites pessoais, páginas comerciais e novas telas da plataforma. Ajuda a transformar um briefing visual em decisões concretas antes de escrever componentes.

Compatibilidade Codex e Claude Code: instruções portáveis. Usa as ferramentas de código e captura de tela disponíveis em cada ambiente.

Exemplo Use $frontend-design para propor e implementar uma página de serviços, preservando a identidade da marca e o conteúdo confirmado.

Limites Não substitui uma pesquisa com usuários nem garante conversão. Imagens, fontes e conteúdo de terceiros exigem suas próprias licenças.

Classificação Fonte oficial da Anthropic, finalidade delimitada e conteúdo inspecionado. A classificação não equivale a uma avaliação de todas as interfaces produzidas.

Pasta da fonte: skills/frontend-design

02 Web Design Guidelines

UX e acessibilidade | Principal | Classificação Alta | Estrelas 32.476

Nome técnico `web-design-guidelines`

vercel-labs/agent-skills

Para que serve Revisa código de interface usando as diretrizes de interfaces web da Vercel. Produz apontamentos ligados aos arquivos e às linhas que precisam de atenção.

Aplicação no meu fluxo Revisão de formulários, navegação e telas existentes nos sites, no CRM e na plataforma. Complementa a criação visual com uma revisão estruturada.

Compatibilidade Codex e Claude Code: instruções portáveis. Precisa acessar a versão atual das diretrizes na web.

Exemplo Use $web-design-guidelines para revisar a tela de cadastro e indicar problemas de acessibilidade e interação com referência ao código.

Limites A revisão do código não demonstra, sozinha, que toda interação funciona no navegador ou com tecnologia assistiva.

Classificação Fonte da Vercel, escopo específico e referência de revisão explícita. Não é uma certificação de conformidade.

Pasta da fonte: skills/web-design-guidelines

Fichas das skills

03 Webapp Testing

Testes funcionais | Principal | Classificação Alta | Estrelas 181.353

Nome técnico `webapp-testing`

anthropics/skills

Para que serve Ajuda a testar aplicações web locais com Python e Playwright. Inclui um auxiliar para iniciar servidores e exemplos para inspecionar elementos, registrar console e capturar telas.

Aplicação no meu fluxo Verificação de formulários e fluxos do CRM, sites e plataforma depois de alterações. Usar dados e ambientes de teste adequados à operação.

Compatibilidade Codex e Claude Code: requer Python, Playwright, navegador e permissão do ambiente para executar o teste.

Exemplo Use $webapp-testing para testar o formulário local, verificar a mensagem de sucesso e registrar a evidência.

Limites Instalar a pasta da skill não instala o Playwright. Um teste sintético local passou nesta configuração; isso não prova o funcionamento de um projeto real.

Classificação Fonte oficial e auxiliar executável. O ambiente de testes foi validado com Chromium, HTTP local, preenchimento, clique, console e captura de tela.

Pasta da fonte: skills/webapp-testing

04 Planning With Files

Planejamento e contexto | Principal | Classificação Média | Estrelas 27.466

Nome técnico `planning-with-files`

OthmanAdi/planning-with-files

Para que serve Mantém plano, descobertas e progresso em arquivos como task_plan.md, findings.md e progress.md. Dá continuidade a tarefas que atravessam sessões ou perdem contexto.

Aplicação no meu fluxo Implementações longas, diagnósticos e refatorações da VouTur e da plataforma. Os registros ajudam a retomar decisões já tomadas.

Compatibilidade O repositório oferece variantes para Codex e Claude Code. A branch usada é master. Os hooks precisam de suporte e configuração próprios do agente.

Exemplo Use $planning-with-files para organizar esta implementação e registrar decisões e próximos passos no projeto.

Limites A instalação feita aqui validou os arquivos, não a execução automática dos hooks. O registro de planos também consome tokens; a economia líquida precisa ser medida.

Classificação Projeto comunitário com recursos específicos por agente. A configuração de hooks e os caminhos exigem atenção adicional.

Pasta da fonte: skills/planning-with-files

Fichas das skills

05 Supply Chain Risk Auditor

Segurança de dependências | Principal | Classificação Média | Estrelas 7.568

Nome técnico `supply-chain-risk-auditor`

trailofbits/skills

Para que serve Analisa riscos nas dependências por metadados de registries, advisories e repositórios. Pode apontar vulnerabilidades, concentração de publicação, abandono e scripts de instalação.

Aplicação no meu fluxo Revisar bibliotecas de sites, CRM e plataforma antes de atualizações ou de uma avaliação de segurança. Complementa a análise do código da aplicação.

Compatibilidade Codex e Claude Code: scripts Python 3.11 ou superior, acesso à rede e GitHub CLI conforme o fluxo de coleta. O auxiliar respondeu a --help nesta instalação.

Exemplo Use $supply-chain-risk-auditor para avaliar as dependências deste projeto e informar o que foi e o que não foi verificado.

Limites Cobre npm, PyPI e Go. O coletor não lê pnpm-lock.yaml, yarn.lock nem poetry.lock. A cobertura e as versões avaliadas devem acompanhar qualquer conclusão.

Classificação Fonte especializada, com escopo e limites explícitos. A classificação considera a configuração necessária e a ausência de uma auditoria completa executada nesta seleção.

Pasta da fonte: plugins/supply-chain-risk-auditor/skills/supply-chain-risk-auditor

06 Improve Codebase Architecture

Arquitetura e manutenção | Principal | Classificação Média | Estrelas 292.556

Nome técnico `improve-codebase-architecture`

mattpocock/skills

Para que serve Explora pontos de atrito na arquitetura e apresenta oportunidades de melhoria em um relatório HTML. Prioriza módulos cuja interface pode esconder melhor a complexidade.

Aplicação no meu fluxo Código difícil de navegar, alterações que afetam muitos arquivos e módulos com responsabilidades mal delimitadas. O objetivo é melhorar manutenção e testabilidade.

Compatibilidade Codex e Claude Code, com ferramentas equivalentes para explorar arquivos e gerar HTML. Usa codebase-design, domain-modeling e grilling.

Exemplo Use $improve-codebase-architecture para analisar o módulo de atendimento e apresentar oportunidades antes de refatorar.

Limites Pode usar subagentes e entrevistas, aumentando tempo e tokens. No Codex, foi removida a chave disable-model-invocation do frontmatter e preservada a política de chamada explícita.

Classificação Fluxo comunitário inspecionado, com dependências e adaptação de metadados. Nenhuma refatoração foi testada apenas por instalar a skill.

Pasta da fonte: skills/engineering/improve-codebase-architecture

Fichas das skills

07 Karpathy Guidelines

Qualidade de implementação | Principal | Classificação Média | Estrelas 220.041

Nome técnico `karpathy-guidelines`

multica-ai/andrej-karpathy-skills

Para que serve Orienta o agente a explicitar hipóteses, evitar complexidade desnecessária, alterar apenas o escopo pedido e definir critérios verificáveis de conclusão.

Aplicação no meu fluxo Correções e refatorações em projetos existentes, especialmente quando há alterações locais que precisam ser preservadas.

Compatibilidade Codex e Claude Code: skill textual, sem serviço externo próprio nem biblioteca adicional para ler as instruções.

Exemplo Use $karpathy-guidelines para corrigir este problema com a menor mudança necessária e verificar o comportamento esperado.

Limites É uma implementação comunitária inspirada em observações de Andrej Karpathy. O nome não significa que a skill seja publicada ou endossada por ele.

Classificação Instruções claras e formato validado, mas efeito sobre qualidade, velocidade e retrabalho depende da tarefa e do modelo.

Pasta da fonte: skills/karpathy-guidelines

08 Caveman

Concisão de respostas | Principal | Classificação Média | Estrelas 112.358

Nome técnico `caveman`

JuliusBrussee/caveman

Para que serve Orienta respostas mais diretas, retirando cerimônia e repetição sem alterar nomes, comandos, números e outros detalhes técnicos.

Aplicação no meu fluxo Tarefas técnicas em que uma resposta curta é suficiente. Preservar explicações completas quando a decisão ou o risco exigir.

Compatibilidade Codex e Claude Code: a versão selecionada é a skill textual básica. Nenhuma CLI, proxy ou conta Caveman foi instalada.

Exemplo Use $caveman e responda de forma concisa, preservando os detalhes necessários. Para encerrar o estilo, diga normal mode.

Limites Não há percentual garantido de economia. O próprio README da skill informa que o resultado depende do modelo e da tarefa; menos texto não mede o custo total da sessão.

Classificação Formato simples e inspecionado, com efeito comportamental ainda experimental. A popularidade do repositório não comprova economia nesta configuração.

Pasta da fonte: skills/caveman

Fichas das skills

09 Git Guardrails para Codex

Proteção do trabalho no Git | Principal | Classificação Média | Estrelas Não se aplica

Nome técnico `git-guardrails`

Original de Matt Pocock que inspirou a adaptação

Para que serve Adaptação local de orientações para conferir repositório, branch e destino do push, preservar mudanças alheias e identificar operações que podem descartar trabalho.

Aplicação no meu fluxo Antes de commits, publicação de branches, reescrita de histórico e limpeza de arquivos. Uma autorização existente para commit e push continua válida.

Compatibilidade Instalada no Codex. O texto pode ser copiado para o Claude Code, mas essa portabilidade não foi testada. Não há interceptador automático de comandos.

Exemplo Use $git-guardrails para revisar as alterações e publicar somente o trabalho autorizado na branch correta.

Limites A versão original para Claude Code instala hooks e bloqueia até git push comum. Esta adaptação fornece instruções e não substitui permissões, revisão ou backups.

Classificação Adaptação curta, inspecionada e validada em formato. Não é uma extensão oficial do Codex e ainda não passou por testes de cenários destrutivos.

Arquivo do kit: skills/git-guardrails/SKILL.md. Estrelas próprias: não se aplica.

10 Codebase Design

Apoio de arquitetura | Dependência | Classificação Média | Estrelas 292.556

Nome técnico `codebase-design`

mattpocock/skills

Para que serve Fornece o vocabulário e os princípios usados na revisão de arquitetura, incluindo interfaces pequenas, complexidade encapsulada e testes pelo contrato do módulo.

Aplicação no meu fluxo Apoio a Improve Codebase Architecture. Ajuda a comparar formas de organizar um módulo e localizar a complexidade.

Compatibilidade Codex e Claude Code: instruções e referências textuais; algumas abordagens descritas usam subagentes.

Exemplo Use $codebase-design para comparar duas interfaces para o módulo de reservas.

Limites Não significa dividir todo arquivo grande. Extrair módulos sem reduzir a complexidade percebida pode piorar a navegação e o custo de contexto.

Classificação Dependência explícita do fluxo de arquitetura, com referências inspecionadas. A técnica deve ser avaliada no código real.

Pasta da fonte: skills/engineering/codebase-design

Fichas das skills

11 Domain Modeling

Terminologia e decisões | Dependência | Classificação Média | Estrelas 292.556

Nome técnico `domain-modeling`

mattpocock/skills

Para que serve Ajuda a manter um glossário do domínio e registrar decisões de arquitetura quando existe uma escolha relevante e duradoura.

Aplicação no meu fluxo Distinguir conceitos como cliente, contato, usuário e agência. Registrar significados compartilhados evita ambiguidades entre código e regras do negócio.

Compatibilidade Codex e Claude Code: trabalha com arquivos de texto como GLOSSARY.md e registros em docs/adr.

Exemplo Use $domain-modeling para esclarecer o significado de reserva e pagamento e atualizar o glossário do projeto.

Limites O glossário não deve virar depósito de detalhes de implementação. Nem toda decisão pequena justifica um registro de arquitetura.

Classificação Apoio específico ao fluxo instalado e formato validado. A correção dos conceitos depende do domínio e da conferência com a implementação.

Pasta da fonte: skills/engineering/domain-modeling

12 Grilling

Revisão de decisões | Dependência | Classificação Média | Estrelas 292.556

Nome técnico `grilling`

mattpocock/skills

Para que serve Estrutura perguntas para testar um plano ou uma decisão. Organiza dependências entre as escolhas e busca esclarecer restrições antes de implementar.

Aplicação no meu fluxo Decisões com alternativas reais, escopo ambíguo ou impacto amplo. Entra como etapa de aprofundamento no fluxo de arquitetura.

Compatibilidade Codex e Claude Code: fluxo conversacional; pode recorrer às ferramentas disponíveis para buscar fatos e explorar código.

Exemplo Use $grilling para revisar as decisões desta mudança de arquitetura antes de implementar.

Limites Pode produzir entrevistas longas. Não é a escolha para correções simples nem uma autorização para reabrir decisões já estabelecidas pelo usuário.

Classificação Dependência do fluxo escolhido, com formato validado. Seu valor depende de usar perguntas apenas onde existe uma decisão pendente.

Pasta da fonte: skills/productivity/grilling

Roteiro de instalação

1 Escolha o destino

Instale Codex ou Claude Code e confirme que o agente funciona. Para os comandos npx, tenha Node.js LTS com npm. Execute node --version e npm --version. Os comandos deste guia são para terminal; no Windows, use PowerShell.

2 Escolha o escopo

Os comandos usam --global para o usuário atual. Remova --global se quiser uma instalação restrita ao projeto. Não use curingas para baixar todas as skills de um repositório.

3 Consulte o que será instalado

Use npx skills add dono/repositorio --list antes de escolher. Confira o nome técnico e a pasta da ficha; um repositório pode conter várias versões da mesma skill.

4 Instale as selecionadas

O arquivo instalar-selecao.ps1 do kit instala somente as onze pastas públicas listadas, mais a adaptação local de Git. Por padrão ele apenas mostra o plano. Use -Aplicar para executar.

5 Abra uma nova sessão

Reabra o chat ou a sessão do agente e confirme a descoberta da skill. No Codex, peça Use $frontend-design; no Claude Code, use /frontend-design. Plugins podem acrescentar um prefixo ao comando.

6 Faça uma verificação proporcional

Peça primeiro que o agente descreva o escopo da skill e confira seus recursos. Depois execute um cenário controlado. Instalação, validação de formato e comprovação funcional são etapas diferentes.

Pastas pessoais e por projeto

Agente	Pessoal	Projeto
Codex	~/.agents/skills/	.agents/skills/
Claude Code	~/.claude/skills/	.claude/skills/

Nesta instalação, o Codex também descobriu skills em ~/.codex/skills/. O kit detecta a seleção existente e evita duplicações. Novas instalações usam o caminho convencional. A estrutura mínima é uma pasta com SKILL.md; scripts, referências e templates são opcionais.

Comandos por conjunto

Alternativa ao kit. Execute apenas os conjuntos desejados. Troque codex por claude-code para mudar o destino. Para os dois, use --agent codex claude-code. Os comandos acompanham as fontes atuais; o kit usa os commits consultados.

C1 Design e testes

```
npx skills add anthropics/skills `
  --skill frontend-design webapp-testing `
  --agent codex --global --copy
```

C2 Revisão de interface

```
npx skills add vercel-labs/agent-skills `
  --skill web-design-guidelines `
  --agent codex --global --copy
```

C3 Planejamento

```
npx skills add OthmanAdi/planning-with-files `
  --skill planning-with-files `
  --agent codex --global --copy
```

C4 Dependências

```
npx skills add trailofbits/skills `
  --skill supply-chain-risk-auditor `
  --agent codex --global --copy
```

C5 Arquitetura e apoios

```
npx skills add mattpocock/skills `
  --skill improve-codebase-architecture codebase-design domain-modeling grilling `
  --agent codex --global --copy
```

C6 Implementação

```
npx skills add multica-ai/andrej-karpathy-skills `
  --skill karpathy-guidelines `
  --agent codex --global --copy
```

C7 Concisão

```
npx skills add JuliusBrussee/caveman `
  --skill caveman `
  --agent codex --global --copy
```

No PowerShell, o acento grave ao fim das duas primeiras linhas continua o comando na linha seguinte; não acrescente espaços depois dele. A versão Markdown oferece cada comando em uma linha. Planning With Files tem variantes por agente; o kit escolhe os caminhos exatos.

Como usar o kit e preparar os testes

Descompacte o kit e abra o PowerShell na pasta extraída. A primeira execução apenas mostra as doze instalações planejadas. Não instala Node.js nem conecta contas.

```
.\instalar-selecao.ps1 -Agente Codex
.\instalar-selecao.ps1 -Agente Codex -Aplicar
.\instalar-selecao.ps1 -Agente Claude -Aplicar
.\instalar-selecao.ps1 -Agente Ambos -Aplicar
```

Escolha uma das três linhas com -Aplicar; não é necessário executar todas. O script recusa pastas já instaladas antes de começar. Não é um atualizador. Se houver bloqueio de execução de scripts, siga a política local sem alterar permissões globais apenas para este kit.

Runtime separado para Webapp Testing

No Windows, numa pasta de ferramentas escolhida por você, crie um ambiente Python e instale o navegador. Use esse Python para os testes; não precisa ativar o ambiente.

```
py -m venv .venv-webapp
.\.venv-webapp\Scripts\python.exe -m pip install playwright
.\.venv-webapp\Scripts\python.exe -m playwright install chromium
```

No Linux, confira as bibliotecas de sistema na documentação oficial do Playwright. O download do navegador exige rede e espaço.

Instalação manual de Git Guardrails

Copie skills/git-guardrails do kit para a pasta pessoal do agente, somente se ela ainda não existir. A pasta contém a adaptação textual usada no Codex. Para Claude Code, a cópia do texto é possível, mas não foi testada aqui.

O bloqueador original do Claude Code é um fluxo diferente: depende de hooks, shell e jq e pode bloquear git push comum. Esta adaptação não instala esse bloqueador.

Ajustes e limites de compatibilidade

Planning With Files

O repositório usa a branch master e oferece uma variante específica para Codex. Instalar a pasta não comprova hooks automáticos. No Windows, confira Python, Git Bash quando um hook chamar sh e os caminhos para scripts. O kit não configura hooks globais.

Arquitetura

As três dependências fazem parte do fluxo. A cópia Codex remove disable-model-invocation do frontmatter e preserva agents/openai.yaml. O kit aplica esse ajuste. Use chamada explícita no Codex. Entrevistas e subagentes podem aumentar o custo; não há promessa de redução de tokens.

Segurança de dependências

A skill da Trail of Bits complementa o Codex Security já disponível. Não substitui análise de autorização, isolamento entre clientes, segredos ou lógica da aplicação. O relatório de dependências precisa informar cobertura, versões e fontes inacessíveis.

Caveman

Foi instalada apenas a skill de concisão. CLI, proxy, roteamento e outros serviços do projeto ficam fora da seleção. Compare consumo e qualidade antes de apresentar qualquer percentual de economia.

Compatibilidade não significa validação completa

Codex e Claude Code compartilham a estrutura de skills, mas diferem em metadados, hooks, ferramentas e políticas. A portabilidade textual foi analisada; a instalação e o teste funcional no Claude Code não foram executados nesta seleção.

Opção que já estava disponível

React Best Practices da Vercel entrou no ranking inicial, mas já estava disponível por um plugin do Codex e não foi reinstalada. A implementação do plugin pode diferir da versão pública. Optimize for Tokens e Plan Token Optimization também já existiam e ficam fora destas doze fichas.

React Best Practices no repositório da Vercel

Verificação e fontes

O que foi comprovado

As doze pastas estão presentes no Codex. A instalação inicial teve integridade conferida; as skills da segunda etapa e o ajuste de Webapp Testing passaram por validação de formato.

Os auxiliares de testes web e de auditoria de dependências responderam a --help.

Playwright e Chromium passaram no cenário sintético local com HTTP, formulário, clique, DOM, console e imagem.

Não foram executados uma auditoria completa, uma refatoração ou testes funcionais de todas as skills. O kit completo e o Claude Code não foram testados por instalação nesta seleção.

Fonte no GitHub	Estrelas	Commit consultado
anthropics/skills	181.353	dbd4588f9e10
vercel-labs/agent-skills	32.476	063bee94c3f4
OthmanAdi/planning-with-files	27.466	96b79bf1d64f
trailofbits/skills	7.568	5c99c03fb8a9
mattpocock/skills	292.556	49dd158d1076
multica-ai/andrej-karpathy-skills	220.041	2c606141936f
JuliusBrussee/caveman	112.358	d262ab388295

Referências de instalação

Skills no Codex

Skills no Claude Code

CLI de skills e opções de agentes

Instalação do Playwright para Python

Links completos de todas as fichas, commits e comandos estão na versão Markdown e no kit. Contagens consultadas em 15/10/2026; estrelas e manutenção mudam com o tempo.

Os projetos pertencem aos respectivos autores. Este guia não implica endosso. Para redistribuir arquivos de terceiros, confira as licenças de cada pasta. O kit distribui a adaptação local de Git e o roteiro, sem copiar os demais projetos.